

CloudCompare: A Cloud Mapping and Migration Analysis Platform

B.Sc. (Hons) Computer Science

School of Computing and Communications
Lancaster University

Matthew Pickard
Student Id: 39009955

Supervisor: Dr Abdessalam Elhabbash

April 2026

Abstract

Cloud computing presents challenges with vast amounts of complicated service data due to provider fragmentation including inconsistent pricing models, varied service families and regional inconsistencies. CloudCompare platform is a platform designed to support cross-provider comparisons and support migration decisions. A custom-built ingestion pipeline, takes raw provider data canonicalising, enabling standardised comparisons to be made. These comparisons give insight into service migration difficulties, allowing users to make informed decisions. An environmental model is factored into the project, providing useful insight into additional considerations required when planning service migration. Evaluation demonstrates the success of components including, performance, ingestion statistics and user feedback which positively suggests that CloudCompare contributes to the cloud migration process.

Declaration

I declare that the material contained in this dissertation is my own and does not contain unreferenced or unacknowledged materials. This statement applies to the implementation of the project and all associated works. Regarding the electronically submitted work, I consent to this being stored electronically and copied for assessment purposes. I agree to my dissertation being placed in the public domain with my name explicitly included as the author of work.

Name: Matthew Pickard

Date: 24/04/2026

Contents

1. Introduction	6
1.1 Motivation	6
1.2 Problem Statement	6
1.3 Aim	7
1.4 Objectives	8
1.5 Contributions	8
1.6 Report Structure	8
1.7 Summary	9
2. Background and Related Work	10
2.1 Growth of Multi-Cloud adoption	10
2.2 Multi-Cloud Migration Challenges	10
2.3 Financial Optimisation and Analysis Tools	12
2.4 Semantic Comparison Approaches	12
2.5 Identified Research Gap	14
3. System Architecture	15
3.1 System Overview	15
3.2 Data Ingestion and Canonicalisation Overview	16
3.3 Service Comparison Overview	18
3.4 Data Transfer and Regional Analysis	18
3.5 Natural Language Processing	19
3.6 Environmental Impact Modelling Overview	20
3.7 Design Limitations	20
4. Technical Implementation	21
4.1 Backend API	21
4.1.1 API Endpoints	21
4.2 Database Implementation	22
4.2.1 Tables	23
4.2.2 Entity Relationship Diagram	25
4.3 Ingestion Implementation	26
4.3.1 Pipeline	26
4.4 Comparison Engine and AI Compatibility	28
4.4.1 Rule-based comparison engine	28
4.4.2 Semantic Similarity with SBERT	29
4.4.3 AI Compatibility Report	30
4.4.4 Limitations	30
4.5 Environmental Impact Model	30
4.5.1 Carbon Emission Model	31

4.5.2 Model Flow Diagram	32
4.5.3 Limitations	34
4.7 Infrastructure	34
4.8 Performance and Optimisation	35
5. Evaluation Framework	37
5.1 Technical Evaluation Design	37
5.2 User-Centred Evaluation	38
6. Results and Evaluation	40
6.1 Ingestion Results	40
6.2 API Performance Results	42
6.4 Environmental Model	46
6.5 User Study Results	49
6.5.1 Qualitative Feedback	51
7. Conclusion and Discussion	52
8. References	54
9. Appendix	56
9.1 Proposal	56
9.2 User Study	63
9.2.3 Consent statements	63
9.2.4 Study link	63

1. Introduction

1.1 Motivation

With the rapid growth of cloud computing across the globe, the demand for multi-cloud solutions has increased drastically, with the majority of large corporations and businesses opting for multi-cloud solutions to help deal with rapid expansion while ensuring cost and performance efficiency across a range of regions.

With this rapid expansion and demand for cloud services comes a massive influx of service offers for a variety of different purposes across hundreds of regions. The three giants of cloud computing providers are Amazon Web Services (AWS), Microsoft Azure and Google Cloud Platform (GCP); each of these provide catalogues containing their service information and pricing data. Service metadata varies in content and format across providers, with large variations making equivalence and surface comparisons incredibly difficult. With a large range of differences in naming conventions, billing units, region availability and configuration data.

Environmental sustainability has become a crucially significant consideration in cloud infrastructure planning. With large corporations being ever more aware of their carbon footprint. Especially with the increase in data centres, with the influx of AI. It is with the climate impact of data centres expected to increase due to rising demand as well as an increase in the carbon footprint by 13% between 2020 and 2030 (*Aslan et al., 2024*). Existing cloud comparison tools primarily prioritise financial optimisation with limited integration of carbon emission modelling or standardised compatibility analysis.

These challenges, both conventional and environmental, expose a gap in how cloud services are evaluated and create the need for a unified cloud comparison framework.

1.2 Problem Statement

Despite the availability of individual provider pricing calculators and cloud financial analysis platforms, standardised cross-provider comparison remains inconsistent and incomplete. Existing tools often emphasise on financial monitoring and physical migration while neglecting the crucial pre-migration analysis, overlooking structured compatibility assessment and software-level equivalence analysis.

Multi-cloud migration involves several variables, including pricing, billing units, regional availability, engine configuration, software versioning, and API level differences. Stark differences in naming conventions, data formats and units further complicate the idea of a standardised comparison model. Cloud provider data is often presented in formats that are difficult to interpret and analyse objectively, often requiring manual research and inspection.

Furthermore, the absence of a standardised model representing canonical cross-provider service data massively complicates this problem and limits the ability of cloud computing customers to understand the fiscal and environmental benefits and limitations of their services. Therefore a unified system is required to standardise this data in order to provide a systematic analysis of fiscal, environmental and compatibility considerations.

In addition to the challenges of data modelling, there is a clear absence of visual analytical tools, especially in areas such as region-based data transfer pricing. Current pricing models and services constrain users' ability to visualise data, confining them to static and extensive tables of data. This form of data representation is monotonous and uninspiring making structured comparison difficult, with reasoning and informative comparisons lost between massive volumes of technical specifications and pricing statistics.

Without the use of appropriate visual elements, crucial factors such as data transfer routes across regions become difficult to interpret and understand. While important regional data is available from some providers, it is once again inconsistent, embedded under different names, service families and scattered across different data structures, making meaningful data extraction a manual and laborious task. This limits the comprehension of prices behaviour and geographical impacts on migration, increasing the complexities of migration analysis.

As well as this, there is a distinct lack of AI-based analysis and reasoning within cloud comparison platforms. While AI-based tools are being increasingly integrated into platforms, they are typically deployed as general assistants providing users with fiscal analytics on owned services rather than for structured migration and compatibility analysis. Cloud providers consistently serve descriptions for service offers; however, these are often limited and do not support structured summarisation of services, leaving uncertainty and the task of service understanding and purpose to the user.

1.3 Aim

This project aimed to design and implement a multi-cloud service mapping and comparison application. The goal of the system is to create a standardised and canonical catalogue of service offers, which can be used in conjunction with comparison engines to support meaningful analysis and generate in-depth analytical compatibility reports.

The platform was designed in order to present this complex service data in a consistent and structured manner to allow for relevant comparisons and conclusions to be made, enabling users to evaluate migration considerations and decisions, assessing all options using the fiscal, regional and compatibility data and reports available to them.

1.4 Objectives

To achieve the aim of this project, the following objectives were set out:

- Develop a cloud data collection system capable of extracting and processing service and pricing metadata from the three main cloud providers using their respective APIs.
- Design and implement a normalised relational database schema to store standardised service information.
- Implement a rule-based cost-comparison model to evaluate services across providers and identify suitable alternatives.
- Integrate semantic compatibility assessment using sentence embedding techniques utilising SBERT model to produce compatibility scores, in conjunction with a thorough compatibility report generated using the OpenAI API.
- Design an interactive dashboard to create analytical and data-driven visualisations of saved service offers.
- Develop a regional data transfer model to visualise data transfer costs across a range of selectable geographical regions.
- Evaluate the system using technical metrics and user studies to assess system performance as well as usability and overall system effectiveness.

1.5 Contributions

This project contributes to addressing the challenges of multi-cloud computing and aims to simplify the complexities of cloud comparison. Some of these contributions include:

- A standardised and normalised data schema for canonicalising cloud metadata and pricing information.
- A hybrid, multi-variable comparison model using variable filtering in conjunction with semantic similarity scoring to assess compatibility between services.
- A data model for storing and presenting the costs of regional data transfers.
- A regional data model for presenting a strong analysis of data transfer costs across and within regions.
- An interactive multi-variable map-based visualisation system providing visual aid to data presentation through the use of geographical data filters.
- An interactive and real-time data dashboard providing fiscal and environmental analysis on compared and saved services.

1.6 Report Structure

The structure of the report is as follows: Chapter 2 covers background research and other related works in the cloud computing industry. Chapter 3 presents the architecture of the system and covers key design decisions and principles, outlining the surface-level workings of the system. Chapter 4 delves deeper into the technical implementation of the platform, detailing and the

inner workings of each system and component of the project. Chapter 5 sets out the framework for the evaluation of the project. Chapter 6 presents the results and their evaluation. Chapter 7 presents the findings and concludes the dissertation.

1.7 Summary

To summarise, this chapter of the dissertation introduced the motivation for the project, setting out the context of the problem and establishing the objectives of the platform.

2. Background and Related Work

2.1 Growth of Multi-Cloud adoption

With the ever increasing demand for web-based products, cloud computing has become the standard for infrastructure. Organizations are more and more likely to opt for Multi-Cloud solutions, distributing their systems across different providers in a variety of regions. This helps to optimise performance and costs based on the requirements of the product. Recent studies indicate that the use of multi-cloud solutions has increased in recent years, with as many as 89% of companies using multi-cloud solutions in 2024. (Flexera, 2024).)

This increased multi-cloud adoption, has led to an influx of new specified cloud services and competitive offers in order to satisfy this large and diverse demand for a range of cloud computing services. While this diversity and range in offers is a brilliant advancement in cloud computing infrastructure, it brings its own complexities and problems, with providers opting for different data structures, pricing models, regional categorisation and service families. While these services broadly perform similarly, these services are not always directly comparable or compatible especially when considering migration options.

The advanced complexity of comparing these cloud service offerings across providers creates difficulties when assessing efficient migration. This division decreases the ability of automated systems for service analysis and requires further manual inspection and documentation, slowing the migration planning process.

With the global usage of Multi-Cloud frameworks and increasing demand for cloud services, comes further environmental concerns and considerations that must be factored into the migration planning process. Data centres are contributing significantly to the global electricity consumption with the global data centre industry expected to emit 2.5 billion metric tons of carbon dioxide by 2030 (*Reuters*, 2024). Due to this, environmental factors are becoming a crucial consideration for customers and providers of cloud services making it an important variable when making cross-platform comparisons or service evaluations. Despite this new push for environmentally friendly computing, there is a lack of platforms providing multi-variable comparisons, including environmental, fiscal and compatibility metrics.

2.2 Multi-Cloud Migration Challenges

There are many complexities involved in Multi-Cloud migration due to the structural incompatibility of the service data offered by providers. Inconsistencies vary across the data, from naming conventions to pricing structures and regional identifiers.

The primary challenge in the area of Multi-Cloud computing is the structural incompatibility of provider API data. This data is typically siloed, with service metadata such as naming conventions, categorisations and pricing data being inconsistent and often incomplete. This complicates the comparison model, requiring extensive mapping, filtering and compatibility logic. The model is limited by the provider with the least available meaningful metadata, as a canonical requires the lowest common denominator between providers in order to provide meaningful comparisons. This issue can be moderately mitigated with the use of provider-only metadata stored in conjunction with the service offered in a JSON structure that allows for more service context and metadata to be provided, depending on availability. While metadata is effective for semantics and input into a large language model, it remains incompatible with a filter-based comparison engine.

Region-code inconsistency further complicates the multi-cloud migration problem, with standardised regions being a critical variable for identifying suitable alternative cloud services. Cloud providers use a selection of different semantic conventions and opt for different levels of detail when categorising geographical regions. Some provide broad geographical regions on the continent-level, while others define regions on specified locations, such as regions or individual cities. These regions are provided in a range of formats, varying greatly in syntax. This massively complicates cost comparison as well as identifying availability in regions.

Pricing structures are also riddled with inconsistencies, due to the complexity of cloud pricing with many varieties in pricing and billing types, with a diverse mixture of reserved, on-demand and commitment offers. The inclusion of development and test consumption offers have potential to skew data, requiring filtering or separate categorisation. Unit inconsistencies across providers require standardisation before conversions can take place with providers opting for a range of different unit nuances and semantics.

Technical incompatibilities are another challenge of cloud-migration, with small or hidden technicalities in services that may appear initially comparable. These may include variations in system architecture, cpu specification, support operating system or engine version. These small but critical nuances can skew the reliability of rule-based filtering better aided by the use of semantic matching techniques and the use of large language models. While all providers, serve information through Web REST API endpoints there are some technical differences in these APIs some serving information in different formats, requiring different authentication methods or pagination that must be traversed. This increases implementation challenges. And a change or update in an API system may require altering or updating in the ingestion pipeline

The combination of these inconsistencies across multi-cloud highlights the challenges of multi-cloud migration and analysis.

2.3 Financial Optimisation and Analysis Tools

In recent years, there has been an emergence of cloud-based analytical tools with a variety of purposes. The vast majority of these being FinOps, aiding in the monitoring, budget tracking and usage analytics of a consumers fleet of services often supporting direct integration with providers to load and analyse current services. These platforms are often very advanced and a great option for providing financial analysis and usage analytics across multiple providers, however focus less on migration compatibility and more fiscal optimisation. FinOps is described as an operational framework and cultural practice which maximises the business value of technology, enabling timely data-driven deciding making and creating financial accountability (FinOps Foundation, 2026). While these tools provide great fiscal importance, critical for businesses, they are the predominant tool in cloud-based analytical tools, reflecting the importance of financial optimisation in the world of cloud computing, indicating that, cloud-based tools focus heavily on fiscal analysis, overlooking compatibility and supporting cross-provider analysis and migration.

Examples of commercially available tools being, VMware's Tanzu CloudHealth, a platform built around the financial analysis and optimisation of cloud services. This commercial software, heavily focuses on providing a range of highly customisable dashboards and economic reports, supporting service deployments and budget tracking (VMware, 2024). Tanzu CloudHealth, further supports environmental impact tracking, offering insight into generated carbon emissions (VMware, 2024). However, despite this high-level of data analysis, Tanzu CloudHealth, demonstrates a lack of compatibility and equivalence tools, focusing on high-level overview rather than specific migration planning. Another commercial tool, Holori is an example of a finance-based cloud analysis platform, providing inventory tracking, economic and cloud optimisation advice (Holori, 2026), as well as standard FinOps tools. However, this platform also lacks in-depth migration planning, reflecting a priority of fiscal tracking. While Holori does provide a comparison calculator (Holori, 2026), providing meaningful fiscal and technological comparisons, it does not support cross-provider compatibility comparisons.

This clearly suggests a gap between fiscal analysis tools and migration planning tools, with existing platforms providing strong fiscal analysis, but failing to provide meaningful cross-provider analysis and comparisons, combining both financial and semantic comparisons .

2.4 Semantic Comparison Approaches

There are many ways to approach the problem of cross-provider comparisons, in cloud computing with traditional approaches such as rule-based filtering, useful for narrowing down potential comparison candidates based on standardised incompatibilities, with key word matching being the standard technique when approaching the cataloguing and comparing of cloud-based services. However, despite this, filter-based matching and key word searching fail to capture the technical specification and real compatibility of services, with services often similar high level,

being entirely incompatible due to technological nuances or software version incompatibility. These issues highlight the importance of contextual understanding when planning cross-provider migration.

Large Language Models, have the ability to provide contextual analysis of service comparisons and compare semantic similarity. However, they are less suitable for a standardised comparison mechanism due to inconsistent outputs, varying in reliability and standardisation. However, being suitable as a supplementary tool, providing comparison-based reports and analysis on already selected comparisons.

Sentence-BERT, a natural language process model proposed by Nils Reimers and Iryna Gurevych (2019). This model converts small amount of text into vectors, with contextual embeddings, allowing similarity between compared texts to be calculated. This produces a number representative of contextual and semantic similarity between two small amounts of text. This method combats traditional key-word matching and rule-based filtering, considering contextual understanding and semantic similarity when comparing service information, enabling deeper analysis of true compatibility between proposed services. This model is very efficient, with improved computational efficiency over alternative model BERT (Reimers and Gurevych, 2019), allowing it to be run on non-powerful machines, making it a suitable comparison tool, used in conjunction with typical rule-based filtering and supplemented by LLM-based reportings to create a full and truthful understanding of the compatibility between proposed services.

2.5 Identified Research Gap

The researched platforms, clearly indicate the abundance of financial modelling and fiscal tools available, with existing platforms providing massive fleets of financial tools, from customised dashboards to advanced financial forecasting models, reflecting the importance and demand for financial monitoring in the field of cloud computing.

Systems or tools that are not financial oriented and are more technically centred, often focus primarily on fleet management and instance deployment rather than migration analysis or planning. These tools are helpful for high-level overview and service monitoring, but lack meaningful depth, especially important when planning the migration of large services. Due to the large variation of properties in services, from billing to regional availability, including regional data transfer costs, to environmental impact, service migration shouldn't be a half-guessed or automated process. The assessment of these components is critical to the smooth transition from service to service, and stresses the demand for an all encompassing, cross-provider migration planning tool.

Existing cross-provider tools do exist, however due to the scale and variation of provider-supplied metadata, they often provide limited migration analysis, focusing primarily on filter-based searching and raw price analysis. This presents a meaningful gap for a platform that provides not only fiscal analysis, but a system that follows the full migration process, providing critical service information, including carbon emissions, regional availability, semantic compatibility while providing detailed AI-generated migration reports. This approach covers all bases required for providing, meaningful cross-provider comparisons while giving insight into the true migration capabilities based on technical nuances and contextual understandings.

3. System Architecture

3.1 System Overview

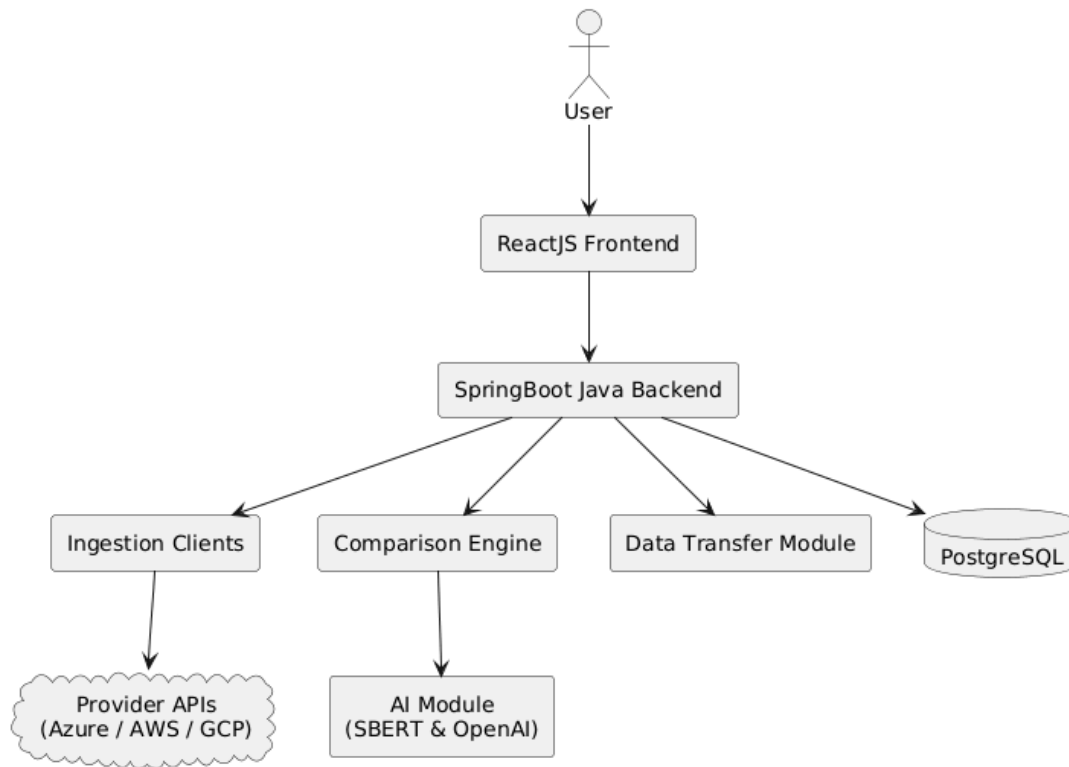
CloudCompare, designed as a modular system, consisting of separated components working in conjunction to meet the objectives of the platform, providing meaningful cloud service offer comparisons, supporting cloud migration. Due to the quantities of data processing required, the chosen architecture was client-server model that separates user interaction from data processing, improving the performance and scalability of the platform, ensuring computationally-heavy tasks are handled separately from the user.

The modular system consists of a web-hosted frontend, a stateless Rest-API, backend data processing system and data storage. These components operate separately but work in conjunction proxied by the REST API, allowing secure communication between modules, while maintaining separation.

The backend system, responsible for handling data, from ingestion management to comparison logic, including required service standardisation for permitting structured comparisons. The backend serves data through web endpoints, providing structured data ready for front-end formatting. This reduces front-end processing as clients are never required to interact with raw ingestion data. The backend stores its canonicalised service data in a relational database, structured and separated schema maximises querying, required for comparisons and data visualisation across large datasets.

This modular designed system maximises performance while limiting client-side processing, ensuring each independent system works at its full potential, to support the CloudCompare system.

Figure 3.1.1 System Architecture diagram



3.2 Data Ingestion and Canonicalisation Overview

Data ingestion forms the backbone of the CloudCompare platform, with accuracy and efficiency of ingestion runs being critical to determine the reliability of generated comparisons and evaluations. Due to the variance of cloud provider data and its lack of standardisation, it is critical a canonical model is used to produce standardised service information suitable for cross-provider comparisons.

Architecturally, the ingestion pipeline is designed to follow a standard flow, retrieving raw service and pricing data, canonicalising important attributes such as pricing, geographical regions and service categories. While cross-provider ingestion follows this pipeline, each provider will have its own implementation due to variations in provider metadata. Due to the variety in data availability and naming conventions, careful mapping must take place to avoid unmapped values, harmful to the comparison tools.

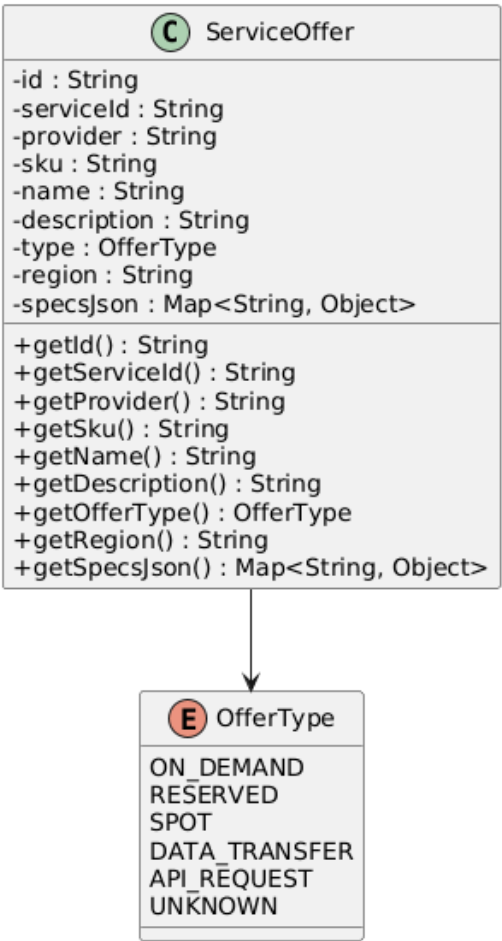
Canoncilisation of the data is critical to the success and accuracy of comparison models, ensuring that the fragmented data can be ingested, formatted and stored in a way that permits

meaningful comparisons through aggressive filtering, important when dealing with large quantities of data and normalised semantic comparisons. Cannocalisation applies to properties such as region, service and offer type, all needed to compare similar services. Additionally, billing normalisation is required, not only with simple unit matching but the mathematical conversion and upscaling of values to fit the standardised model required for accurate price-depicting comparisons. Further data, not fitting of the cannoncalised model is to be stored separately allowing for later semantic contextual consideration and comparison, this data includes; technical specifications, non-cannonical metadata and descriptions of the product.

Furthermore, data ingestion architecture is designed to be tracked and monitored in real-time providing insights on live errors, ingestion throughput and distribution of offer types. This data is critical for not only the evaluation of the platform, but for the scalability and maintainability of the ingestion framework.

Figure 3.2.1 Cannoncalised service offer class diagram

Canonicalised Service Offer Data Model



3.3 Service Comparison Overview

The service comparison module of CloudCompare, adopts a hybrid system, combining both rule-based filtering and semantic similarity. This dual system, covers critical factors of cross-provider comparisons, these being standardised similarities and semantic compatibility.

Rule-based filtering, treated as the primary comparison model, responsible for comparing canonical attributes in the effort to remove redundant candidates, filtering out millions of service offers, to isolate only structurally comparable services. This aggressive filtering is crucial performance, due to the large quantities of available cloud data, finding services that are similar in both, region, pricing, service family is incredibly important for providing meaningful comparisons. The rule-based comparison model is designed to produce a selection of alternative offers ascending in price, highlighting the cheapest alternatives. However, this does not indicate true compatibility, with technical nuances being irrelevant to rule-based filtering, preventing proper migration or fair comparisons.

The proposed SBERT model provides additional semantic similarity scoring, this scores compared services based on contextual similarity rather than identical attributes. These scores used in conjunction, with supplementary AI-generated reports, enable deeper compatibility insights as well as migration difficulty analysis, otherwise impossible with rule-based filtering. The AI-based report is treated as a truly supportive tool and is not deemed reliable as a core comparative method.

Overall, the architecture of the comparison model, utilises multiple methods to produce compatible and meaningful insights into compared cloud services.

3.4 Data Transfer and Regional Analysis

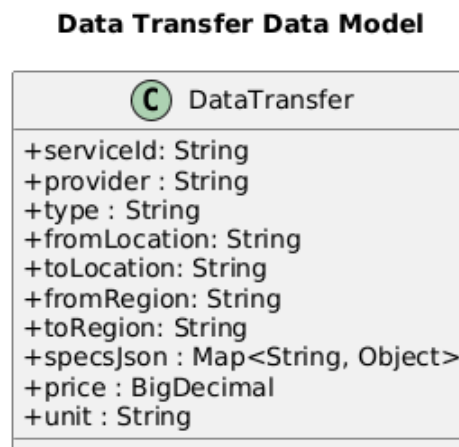
CloudCompare is designed to handle and present regional data transfer costs, providing graphical and data-driven understanding of what is often an overlooked cost in the world of cloud computing.

Data transfer costs are often fragmentally scattered across provider information with a range of representations and crucial location information is often hidden amongst insignificant metadata. Due to this, the data transfer model on the CloudCompare platform is not cross provider, due to the availability of regional location and destination data, Azure is the only suitable provider for information pertaining to data transfers. Regardless, this information is crucial and incredibly useful for analysis.

Due to the data structure differences involved in data transfer services, as opposed to typical service offers. CloudCompare opts to isolate these, storing data transfers in a separate data

model, this isolation prevents canonicalisation formalities allowing on the necessary and relevant data to be captured. Unlike typical service offers, data transfer offers store information pertaining to origin and destination locations, required for graphical mapping and true contextual understanding. Furthermore, these offers have canonicalised types, relevant to the type of data transfer being carried out. The figure below shows the opted for, data transfer model.

Figure 3.4.1 Data Transfer class diagram



Overall, data transfer costs and its visualisation within the CloudCompare platform, provides meaningful insight into the additional costs that may influence migration decisions.

3.5 Natural Language Processing

Natural Language Processing, was a chosen methodology for non rule-based comparisons as the project aimed to provide contextual understanding that was often missing from existing systems. Sentence-BERT was the chosen method for this, due to its ability to capture semantic and contextual understanding, and ability to prove similarity between contextually similar services, described in a textually different way. Other models such as key word matching or TF-IDF were considered, but disregarded due to a lack of contextual understanding and an emphasis on repeated key word comparisons.

This makes SBERT an appropriate choice, especially in conjunction with a filter-based system to ensure that the compared services are structurally compatible, as SBERT benefits from utilising standardised formatting, improving reliability of similarity scores.

3.6 Environmental Impact Modelling Overview

CloudComparison offers environmental impact metrics for related cloud services, implementing an environmental emission model that utilises service metadata such as region, provider and type, as well as externally gathered environmental statistical data. This metadata is used to estimate the environmental impact of cloud services, supporting service comparisons and creating meaningful dashboard insights on saved services.

The system does not opt for exact emission measurements, but rather accurate estimates based on consistent variables used across the comparison pipeline, thus supporting users' migration decisions, considering environmental trade-offs in conjunction with other fiscal and compatibility metrics.

3.7 Design Limitations

Due to varying factors, there are several limitations to the architecture of the CloudCompare platform, due to the dependence on third-party service offer data, the data model must be designed to incorporate data consistent across all providers, resulting to the loss of service data that does not fit the canonicalised model, while provider specific metadata is stored separately, filter-based comparisons can only be made on the lowest common denominator structure of data available.

Furthermore, while useful as a supplementary tool, Sentence-BERT commonly misses nuances in technical and architectural configurations of service offers, making it not entirely reliable for full migration but rather a supplementary tool to support understanding. AI-generated reports are, similarly flawed, designed as an insight tool rather than a fundamental comparison system, due to variance in outputs and reliability.

The environmental component, utilises a simplified emission model that calculates data based on world averages and historic data, rather than live-updates. Therefore, outputs should be considered as additional insight information designed to aid migration decisions rather than to be interpreted as precise data.

In conclusion, the limitations of this CloudCompare's architecture highlight its purpose for migration planning and insight, rather than system management and migration implementation.

4. Technical Implementation

4.1 Backend API

The backend API was a critical design element, responsible for serving all processed offer metadata and corresponding comparisons and reports. The backend API was implemented with SpringBoot in Java, providing a modular framework, permitting a stateless design, enabling asynchronous requests, prioritising performance and efficiency. The separation of data processing logic and the REST API enabled the implementation of a modular system.

API endpoints are implemented for each component of CloudCompare. This enables isolated data retrieval per component, improving maintainability and future compatibility. This modular design was further supported by the use of DTOs for large data payloads, increasing robustness and forward-compatibility.

The use of Spring Boot was critical, hailed for its simplified configuration and built-in features; it was the perfect framework to build CloudCompare's controller layer upon, permitting parameterised queries, crucial for interacting with CloudCompare's comparison modules. Security was another critical consideration, and with Spring Security, JWT authentication was implemented, allowing access to restricted endpoints with the use of a generated web token. This enabled a permission system to be established, permitting different levels of access dependent on the active user account.

4.1.1 API Endpoints

The table below provides information on the key endpoints in the CloudCompare system. There are more endpoints, but the number has been reduced for this report.

Figure 4.1.1 Key Endpoints

Endpoint	Method	Description	Required Role
/auth/register	POST	Registers a new account	N/A
/auth/login	POST	Authenticates the user and returns a JWT	N/A
/offers/map	GET	Returns aggregated map data	User
/compare/{offerId}	GET	Returns comparison results for a select offer	User
/compare/ai/{offerId}?offer2Id={offerId2}	GET	Runs and returns an AI comparison report.	User
/emissions/{offerId}	GET	Returns the emissions of a specified offer	User
/saved	GET	Returns a user's saved offers and pertaining metadata	User
/admin/status	GET	Returns ingestion statistics and dashboard information	Admin
/admin/ingest/...	GET	Triggers a provider-specific ingestion	Dev

4.2 Database Implementation

CloudCompare, by design, must process and store large quantities of data, canonicalising and upserting millions of offers in a matter of hours. Due to the large volume of data and specified comparative tools, the implementation of databases is a critical design decision.

Due to the data comparison model required, a relational design was chosen.. This supported data integrity, providing a normalised structure for the data. The database schema was designed with normalisation in mind, reducing information redundancy and improving efficiency. The separation of pricing and service data improved query performance while reducing the risk of duplicate records. Furthermore, this schema supported raw service metadata being stored in JSONB, enabling data flexibility for non-canonicalised models such as SBERT or AI analysis. This normalisation-centred design provides maintainability support as well as forward

development, supporting further providers, additional data and providing future support for new models or re-ingestion.

The database schema was initialised inside the backend within a *TableInitialiser* module, responsible for the creation of tables. This was done on startup with the Spring Boot, `@PostConstruct` method, ensuring creation of the tables before any operations are executed. PostgreSQL was the chosen database engine due to its improved performance compared to alternatives, while providing JSONB support, which is critical for the hybrid, structured but flexible data model.

4.2.1 Tables

Services - Stores canonical services categorised (e.g. compute, storage, database). This enables valid comparisons between services.

Service_offers - Stores services offered from providers, containing all relevant offer information used for comparisons. Separated to allow for individual offer implementations rather than set per service type.

Prices - Stores pricing information for service offers, used for offer comparisons. This information is stored separately from offers to improve querying performance and reduce redundant and duplicate data.

Network_transfers - Stores mapped network transfer costs, including information on origin and destination locations, as well as relevant pricing data. Stored in isolation from standard service offers due to its use in separate map visualisations.

Users - Stores information regarding user-created accounts, which dictate endpoint authority and are critical in their use of the service, saving for comparisons.

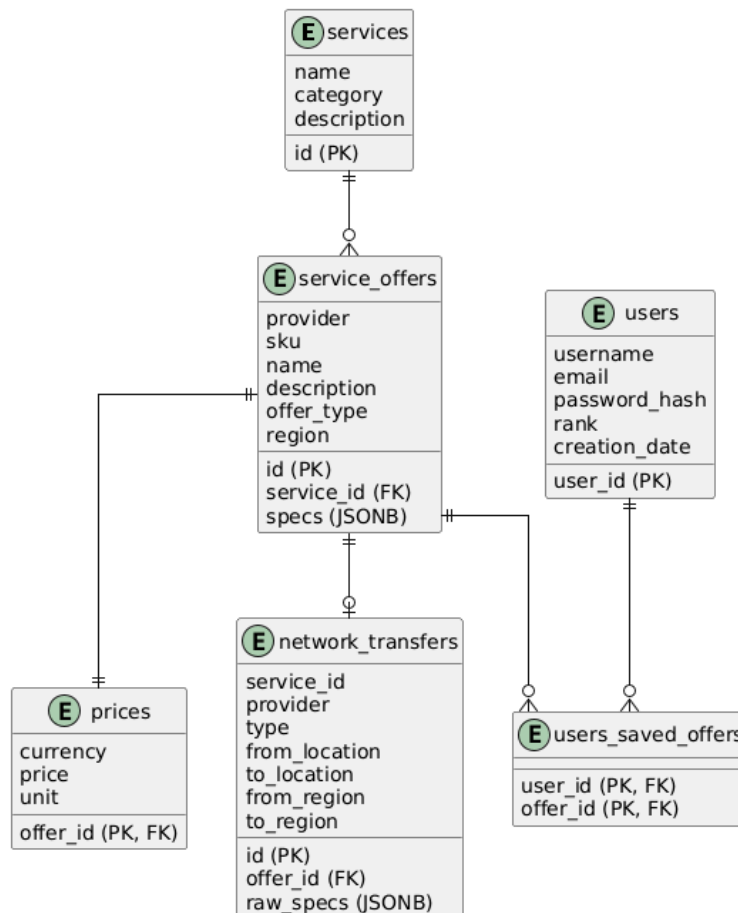
User_saved_offers - Junction table, mapping saved service offers to user accounts, isolated for efficient offer retrieval, especially if the selection of saved offers is large.

Figure 4.2.1 Table Design

Table	Field	Field Type	Description
services	id (PK)	TEXT	Identifier for service type
	name	TEXT	Service name
	category	TEXT	Service category type
	description	TEXT	Detailed service description.
service_offers	id (PK)	TEXT	Identifier for a service offer.
	service_id (FK)	TEXT	References specific service.
	provider	TEXT	Service Cloud provider
	sku	TEXT	Provider SKU id
	name	TEXT	Offer name
	description	TEXT	Offer description
	offer_type	TEXT	Canonicalised offer type
	region	TEXT	Canonicalised region
	specs	JSONB	Service offers JSON metadata
prices	offer_id (FK)	TEXT	References associated offer id
	currency	TEXT	Associated currency code
	price	NUMERIC	Offer price
	unit	TEXT	Offer pricing unit
network_transfers	id (PK)	BIGSERIAL	Unique identifier for data transfer
	offer_id (FK)	TEXT	References offer id.
	service_id	TEXT	Service type
	provider	TEXT	Cloud provider
	type	TEXT	Dictates type of network transfer
	from_location	TEXT	Origin location
	to_location	TEXT	Destination location
	from_region	TEXT	Canonicalised origin
	to_region	TEXT	Canonicalised destination
	raw_specs	JSONB	Raw offer & transfer metadata
users	user_id (PK)	UUID	Unique identifier for username

	username	TEXT	Username
	email	TEXT	Email address
	password_hash	TEXT	Hashed password
	rank	TEXT	User's rank/role
	creation_date	TIMESTAMP	Date of account creation
users_saved_offers	user_id (PK/FK)	UUID	References the user account id
	offer_id (PK/FK)	TEXT	References offer id

4.2.2 Entity Relationship Diagram



4.3 Ingestion Implementation

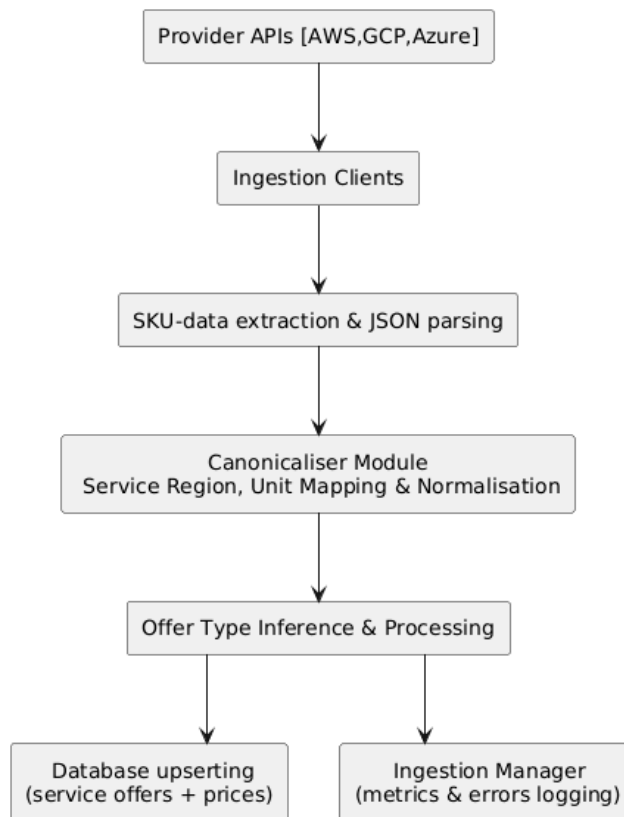
The ingestion pipeline is arguably the crux of the CloudCompare platform, responsible for retrieving, filtering and canonicalising millions of service offers, from a range of diverse and often fragmented cloud data; consequently, the implementation and efficiency of the pipeline is critical. The ingestion framework prioritised efficiency and error handling, ensuring the highest yield of service offers, utilising modular components, permitting per-provider implementation and supporting further extensibility for new provider implementations.

Despite the modular nature of the ingestion framework, all ingestion clients follow the same base pipeline, differing only by data availability and provider-specific semantics.

4.3.1 Pipeline

Ingestion clients follow a set ingestion pipeline, which retrieves, processes and transforms raw service metadata into canonical service offers. Regardless of implementation specifications, this abstracted pipeline outlines the process the data undergoes before being used in comparative models. An abstract pipeline diagram is provided below.

Figure 4.3.1 Ingestion Pipeline Overview



Multiple stages are involved in the ingestion process, from data retrieval to ingestion tracking. These stages are as follows:

1. API Retrieval

The first step of the ingestion pipeline is gathering raw servicedata from cloud provider endpoints, this data is requested through HTTP Clients, saving raw bytes before converting them to JsonObjects for field extraction.

2. SKU Field extraction

The returned JsonObjects are then explored, searching for key words, utilising mappings to extract the required fields for canonicalisation. Non-cannonical fields are added to a specifications list. These retrieved fields are cleaned, formatted and stored.

3. Service Mapping

Stored attributes are used to map offers to specific services such as (**svc-compute**, **svc-mysql-db**, **svc-storage**). Offers that do not fit into the categorical types, are discarded as they do not fit the data model.

4. Region Mapping

Raw region data, from SKU field extraction are passed through a Region Canonicaliser module, this contains a database of raw region names that map to a canonical region. Unknown regions are logged to a local file for later analysis.

5. Unit Normalisation

Billing units vary drastically across provider exposed data, normalising this is critical for the comparison engine to produce meaningful results. Unit normalisation is also implemented through rule-based mapping tables, standardising raw units and upscaling and downscaling prices where required.

6. Offer Type Mapping

Offers vary greatly in type and only offers of the same type must be compared, required raw offer units to be mapped to a standardised system. The cannoncalised types are as follow; *ON_DEMAND*, *RESERVED*, *SPOT*, *DATA_TRANSFER*, *API_REQUEST* and a fallback type of *UNKNOWN* is applied in the event offer mapping fails.

7. Optional Technical Specifications

In the case that metadata does not fit the canonical model, instead of being disregarded, some of this data is stored as flexible json, preserving raw ingestion data as well as additional technical specifications, useful for prompting LLM generated reports.

8. Database upserting

Once data has been extracted and standardised, a unique identifier is generated as such:

offerId = "offer-provider-" + formattedRegion + "-" + Type + "-" + SKU;

This helps preventing duplicate records, and is the primary identifier of saved offers. The service metadata is upserted into the database along with this identifier.

9. Ingestion tracking

During this entire process, an *IngestionManager* module, handles individual Ingestion processes, logging ingestion throughput, errors, offer type distribution and other metadata for later analysis and live tracking.

4.4 Comparison Engine and AI Compatibility

CloudCompare's comparison engine is implemented using a combination of rule-based filtering and semantic compatibility checking. This system includes the generation of a compatibility evaluation supported by semantic verification, contextual reports and insights.

4.4.1 Rule-based comparison engine

The rule-based comparison is the primary mechanism in the migration pipeline, making use of query constraints and enforcing alignment of set canonicalised metadata. The compared services use these canonical variables, such as service family, region, offer type and billing unit. These constraints ensure that the evaluated services are structurally comparable, preventing incompatible alternatives from being suggested.

This rule-based filtering is implemented through parameterised SQL queries. These allow for incredibly fast retrieval times, especially with a large number of potential candidates. After filtering has been applied, services are ordered by cost in ascending order, displaying the most cost-effective alternatives to the user.

The parameterised filtering model supports optional parameters, which allow queries to be dynamically altered according to user preference. Additionally, SQL techniques such as *LIMIT* and *OFFSET* are used to implement pagination for performance over large sets of data. Pricing data is joined with service information to provide appropriate price ordering, reducing the processing demands of the front-end application logic.

4.4.2 Semantic Similarity with SBERT

Semantic Similarity is the secondary module in the comparison framework, implemented in the backend engine (*SbertEngine*). This operates in addition to rule-based filtering.

CloudCompare implements SBERT through an optimised ONNX model. The ll-MiniLM-L6-v2 model and HuggingFace tokeniser are initialised and loaded on startup using the `@PostConstruct` method.

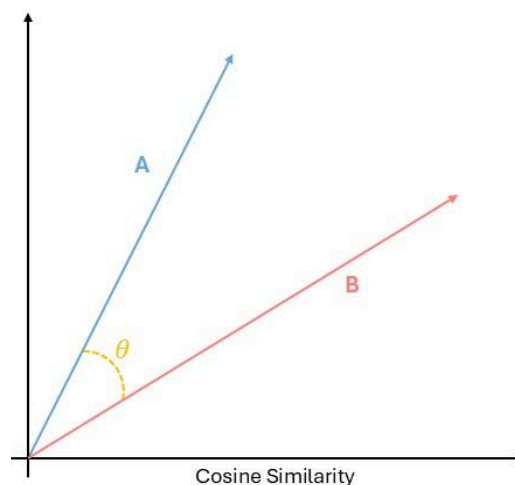
Selected Service metadata is filtered, isolating specific attributes for text normalisation. The *sbertTextNormaliser* module converts these isolated attributes, such as name, description, offer type and other metadata, producing one string, allowing the model to capture the semantic context of the Service's technical and descriptive properties. These produced strings are tokenised before being passed through the model to produce token embeddings.

Before the similarity between embedding vectors can be calculated, pooling must be applied. Mean pooling is a method used in sentence transformers to obtain an average of vector embeddings. This method prioritised computational efficiency and simplicity rather than deep semantic understanding and word order analysis, as required by the platform. The result is a fixed-length embedding vector.

Similarity between produced embedding vectors is then computed using cosine similarity:

$$\text{Similarity} = \cos() = \frac{(A \cdot B)}{(\|A\| \|B\|)}$$

A and B are the normalised embedding vectors, and measure the angle between them to produce a similarity value. A visualisation is provided below.



4.4.3 AI Compatibility Report

In addition to the filter-based and semantic comparison models, CloudCompare generates comparative and migration reports with the use of a Large Language Model. OpenAI's gpt-5-mini is the chosen model, implemented through the OpenAI API.

A specialised prompt is fed to the model in addition to corresponding service metadata, to the OpenAPI endpoint through a purpose-built HTTP client. This produces a report in a structured JSON format. The report, designed as a supplementary contextual tool, provides meaningful descriptions and comparisons of selected services, highlighting similarities, differences and emphasising migration difficulty.

4.4.4 Limitations

There are limitations to the AI and Semantic comparison components implemented within CloudCompare, with heavy reliance on external services, generating tailored comparison reports comes with additional latency and network traffic alongside fiscal requirements of Large Language Model provider, OpenAI. Due to the scope of the project, this was deemed the best implementation of this feature with the trade off of performance.

Furthermore, the implementation of Sentence-BERT, while insightful for the comparison of similar service types, important nuances such as technological specifications are often lost due to the over generalisation caused by issues such as; token restriction and embedding averaging techniques. Mean pooling was used to average the tokenised embeddings, producing a single vector, prioritising efficiency yet sacrificing contextual information and token ordering. This is impactful due to the large number of similar service offers, slightly differing in technical or semantic specification but largely differing in operation, performance creating unbeknownst migration challenges. While these limitations are real, they are tradeoffs for a platform prioritising performance, treating these semantic, ai components as simply supplementary tools, rather than core comparison fundamentals.

4.5 Environmental Impact Model

CloudCompare implements an environmental impact model, estimating emissions produced by catalogued cloud services. This model is run on service offers, within the backend system and directly integrated into both catalogue system and comparison pipeline, allowing emissions to be calculated for any identifiable service offer. This model provides emission data for individual services, as well as calculating aggregation data permitting graphical map insights. This is handled by the *EmissionManager* module.

4.5.1 Carbon Emission Model

Scope 2 emissions refer to the greenhouse gas emissions produced by electricity, with Scope 2 representing one of the largest sources of greenhouse gases emissions globally (GHG Protocol, 2015). Our model estimates these Scope 2 emissions, calculating the indirect greenhouse gases produced by the electricity consumed by the service providers included in the CloudCompare platform. The equation for this model is as follows:

$$CO_2 (kg/h) = Watts \times PUE \times (Grid Intensity / 1,000,000)$$

The equation calculates Co2 emissions by combining service power usage, data centre efficiency (PUE) and the region's grid carbon intensity. This formula estimates emissions in grams of CO2 per hour which we then divide to convert it to kilograms an hour. Once applied, it generates estimates of a service's average emissions and an efficiency rating from A-E is given.

The three parameters used in this model being; approximate service energy usage, regional emission factor and provider power efficiency (PUE). These values are used to calculate standardised, comparable emission estimates across providers and differing service types.

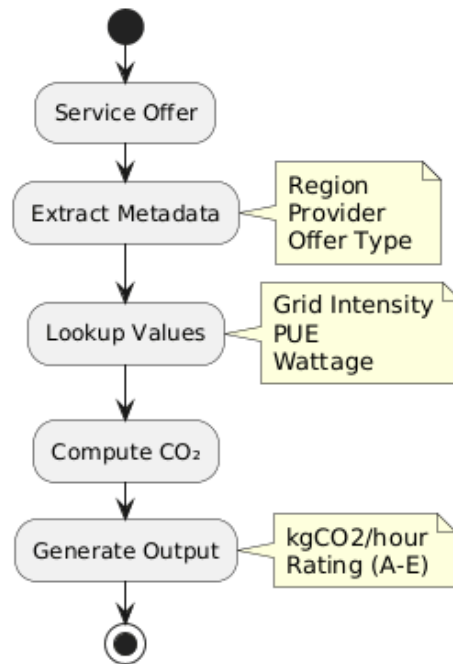
Regional emission factor, represents the carbon intensity requirements of a region (gCo2/kWh). This indicates the cleanliness of the regional power grid, with variations in sources of electricity generation. Regions with higher reliance on non-renewables such as fossil fuels, exhibit higher carbon intensity values, whereas regions such as EU-NORTH, with higher reliance on renewables would exhibit a much lower carbon intensity. These regional carbon intensity values are assigned to canonical regions used inside the CloudCompare data model. These grid intensity values are sourced from a variety of datasets provided by organisations such as the IEA, UK Grid data and EPA's eGrid database. (IEA, 2024; National Grid ESO, 2023; EPA, 2022). These values are representative of regional data and where unmapped, a default average value of 445gCo2/kWh is applied.

Provider's datacentre power efficiency is incorporated using Power Usage Efficiency (PUE). A PUE value closer to 1 indicates less energy overhead suggesting better efficiency. The values pertain to the individual efficiency of providers, gathered from sustainability policies (AWS, 2024; Google, 2024, Microsoft 2022). An average value of 1.20 was applied for services with no mapped provider.

Service energy consumption is indicative of the average wattage consumed by differing service types within cloud computing. These values were gathered from The Carbon Footprint (CCF)

methodology, an tool that estimates emissions of cloud services, utilising SPECpower benchmarking data. These produced power consumption values, can be used by the model to calculate estimated carbon emissions for services with varying types. Thoughtworks, 2021; SPEC, 2018).

4.5.2 Model Flow Diagram



Provider	Power Usage Effectiveness (PUE)
AZURE	1.18
AWS	1.15
GCP	1.10
Default	1.20

Region	Regional Carbon Intensity (gCo2/kWh)
EU-NORTH	46
CANADA	110
UK	178
EU-WEST	218
US-WEST	248
EU-SOUTH	280
EU-CENTRAL	340
US-EAST	386
US-CENTRAL	450
ASIA-EAST	555
AFRICA	598
ASIA-SOUTHEAST	600
ASIA-SOUTH	708

Service Type	Service Energy Consumption (watt-hours)
ON_DEMAND	150
RESERVED	150
SPOT	150
DATA_TRANSFER	5
API_REQUEST	2
OTHER	50

4.5.3 Limitations

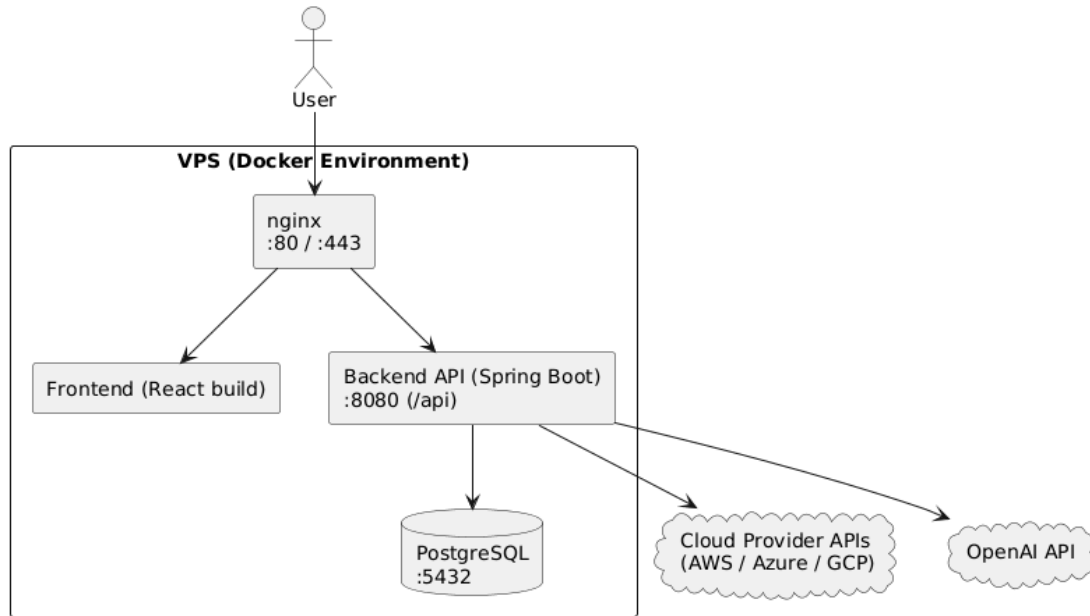
Due to the time constraints and scope of this project, a simplified carbon emission model was chosen, opting for simple yet meaningful predictions rather than perfect calculation. This simplified model utilised fixed values based on public data and national averages, it fails to account for real-time variation, due to grid intensity or specific service workload estimations, which would improve the model yet drastically increasing both computational workload and complexity. Regardless of these limitations or areas of improvement, the simplified model serves its purpose, providing contextually relevant emission data useful for cross-provider comparison and migration decisions.

4.7 Infrastructure

CloudCompare was designed and deployed with containerisation in mind, with large data and ingestion requirements, scalability and infrastructure were key considerations. Deployment is executed using Docker Compose, housing three main components: a PostgreSQL container, a Java Spring Boot backend container and an nginx container, serving the frontend of the project and handling routing requests. These compartmentalised modules are secured, running isolated services while still utilising safe communication across a secured Linux VPS.

With CloudCompare being a data-oriented system utilising web endpoints that serve critical data sets, security is crucial. Sensitive information, such as API keys, is stored as environment variables alongside critical data information such as: datasources, ports, usernames and passwords. This data is stored in an environment file, which is injected into the software on startup. CloudCompare serves its own data through REST APIs via SpringBoot. Security and access to these endpoints are controlled through an authenticator provided by JWT, ensuring that only authenticated users can make requests to the endpoints, protecting them from malicious use. Critical design principles of stateless deployment and service separation are used throughout the deployment and infrastructure of the project.

Figure 4.7.1 Infrastructure diagram



4.8 Performance and Optimisation

Due to the nature and quantity of data, it is designed to be processed by the CloudCompare system. Optimisation and maximising performance were critical design decisions. Performance techniques were applied across the system, critically at the catalogue level, providing and serving millions of service offers, requiring efficient pagination, implemented with *LIMIT* and *OFFSET*, reducing network traffic, database load and response time.

Pagination and filtering are implemented via parameterised SQL statements, with filtering applied using optional constraints such as provider, region, offer type, etc. This prevents the retrieval of redundant information, improving both memory efficiency and retrieval times. Further optimisation techniques are applied, such as indexing frequently fetched fields, such as identifiers, allowing the database to locate the corresponding records without performing a full search. This improves efficiency, especially with queries on large datasets.

Furthermore, in aid of dashboard and analytical map tools, aggregation queries are executed on the backend database to save front-end processing power, calculating averages and counts across service information and regions, enabling quicker response times and less processing overhead.

Performance metrics are measured during ingestion, a critical component of CloudCompare. These metrics include execution duration, number of offers processed, success and error rates. This enables the identification of performance issues. A further logging system for unmapped regions helps improve the mapping system, increasing the number of mapped services and reducing errors.

5. Evaluation Framework

The evaluation of CloudCompare must assess both the usability of the CloudCompare platform as well as the technical capabilities of the system it is built upon. Due to the scale and quantity of components involved in both the backend system and user-available interface, there are many aspects to evaluate including performance, data model accuracy and usability.

The evaluation aims to give insight into the overall effectiveness of the platform, providing qualitative and quantitative data regarding the overall effectiveness of the CloudCompare platform across many metrics.

The key objectives for the evaluation are as follows:

- To validate the accuracy of results produced by the carbon emission model.
- To measure performance across critical system endpoints, with varying conditions and parameters.
- To evaluate the effectiveness of semantic comparisons on computed service descriptions, to dictate real compatibility.
- To evaluate the usability of the CloudComparison website through user provided likert scales and qualitative feedback.

5.1 Technical Evaluation Design

Technical evaluation was completed entirely through the use of CloudCompare's API acting as a proxy, providing insights into the technical metrics of the system in live time, this permitted detailed data gathering during the running of technical pipelines and operations from; live ingestion data to aggregative map queries.

System performance was evaluated through endpoint response time, measured in milliseconds (ms); these tests were run systematically on every critical endpoint, categorised based on function, with categories expected to differ in performance due to computational requirements. Simple queries; such as authentication and basic data retrieval were expected to have fast response times, while aggregated and filtered queries were expected to experience slower times. These system tests were carried out in order to ensure performance was as expected in practice.

Furthermore, technical evaluation was designed to encompass the critical ingestion pipeline, responsible for processing millions of offers an hour, it was crucial that the evaluation covered both the efficiency and reliability of this pipeline, due to its importance to the data model. CloudCompare has a built-in ingestion monitor, accessible via API, allowing for the monitoring and gathering of ingestion statistics critical for evaluation.

Carbon Modelling is a critical component of the technical evaluation, testing the implemented carbon emission model, ensuring accurate and valid results that give meaningful insight relevant

to comparisons. This model would be tested utilising varying variables such as; provider, service type and region in order to identify which factors have the greatest effect on emission predictions.

The evaluation of semantic comparisons, implemented utilising SBERT, are a key component of the evaluation design. Evaluating the clarity of the semantic scoring system, provides understanding of the effectiveness of the comparison feature beyond core filter-based comparisons.

5.2 User-Centred Evaluation

Despite the evaluation being technically centred, the user evaluation was also critical for assessing the usability and practicality of the platform. Due to the website being the primary interface of the project it makes it equally as important as the correctness of backend systems.

The design of the user study was a pivotal consideration, aiming to gather both quantitative and qualitative data. Designed in the form of an anonymous study containing guided tasks in which users are required to identify and locate services, executing comparisons. This study was designed to assess the user's interactions across every part of the system including, authentication, the service catalogue, the comparison pipeline, dashboards and administrative analytical tools.

On completion of the study, participants are prompted to complete a structured feedback form, this provides users with likert-style questions relating to the tasks they have completed, indicating the ease of completion. Additionally, open-ended questions were provided to gather not only task metrics but qualitative information on user experience and system ease of use. Open ended questions were marked as optional due to their variance in content and application to areas of expertise. These questions enable the analysis of task completion, and provide insight into features or components of the platform found favourable by users.

The study was targeted towards a wide range of individuals, across varying relevant fields such as computing, cloud engineering and data science. A range of experience across selected participants allowed the evaluation of varying components offered by CloudCompare, with data-centred participants having greater comprehension of service offer dashboards and ingestion-related data. These users with experience in cloud computing services, were more likely to understand the expected results of comparisons and properly evaluate the effectiveness of the comparative tools and models.

Users with less experience, such as computer science students and graduates were also critical to this with any website, usability and intuition is important, especially when the tool aims to present complicated technical data in a simplified and understandable format. Due to this, it is important for CloudCompare, a decision support platform, to be easily navigated, inferred and utilised by users with a range of experience without existing knowledge of the system. The feedback provided by participants with a range of experience and prior knowledge will help to build an idea of the usability of the system and what the strengths and weaknesses of the system and its interface are.

6. Results and Evaluation

6.1 Ingestion Results

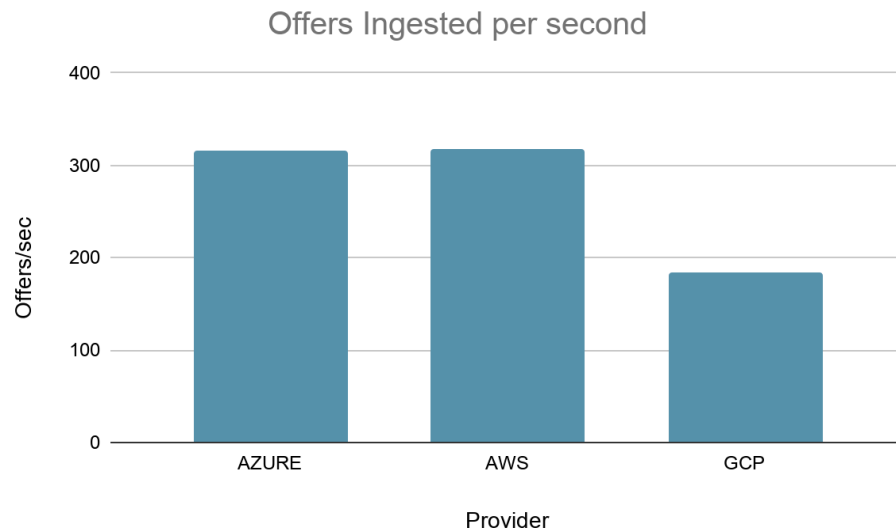
The ingestion pipeline was evaluated based on ingestion statistics gathered by the *IngestionManager* module, this tracked successful and failed ingestions over the course of the pipeline process, logging duration and ingestion speed in addition to errors incurred. The raw ingestion results are shown in the figure below.

Figure 6.1.1 Ingestion Results

Provider	Ingested Offers	Duration (seconds)	Offers/Sec	Errors
AZURE	724,979	2293	316	233,203
AWS	530,747	1671	317	0
GCP	45,136	159	184	0

The ingestion results show results from each of the three providers; with Azure obtaining the highest number of ingested offers, followed by AWS and GCP. While Azure dropped $\frac{1}{4}$ of ingestions due to errors, these are typically ingestion incompatibilities rather than serious system errors and differ to the zero errors incurred by AWS and GCP due to differences in the pipeline's implementation, restricted by the semantic and architectural difference in both the cloud provider's data and the data source. The following figure dictates the offers ingested per second, calculated by dividing total offers by total duration.

Figure 6.1.2 Ingestion Rate



As shown in figure 6.1.2, both Azure and AWS averaged over 300 ingestions per second, while GCP came in much lower at 184 due to advanced processing required by GCP the ingestion client, due to the nature and complexity of GCP-provided service data.

Further evaluation metrics included, ingestion offer type distribution. This enabled insight into the type of data ingested and what proportion of commercially available cloud services fall into what category of computing type. This data is presented in the figure below.

Figure 6.1.3 Offer Type Distribution

Offer Type	AWS	AZURE	GCP	TOTAL
ON_DEMAND	512,182	571,813	25,451	1,109,446
RESERVED	0	158,372	4,952	163,324
SPOT	0	0	3,313	3,313
DATA_TRASNFER	296	121	17,368	17,785
API_REQUEST	686	0	108	794
UNKNOWN	17,584	0	478	18,062

As clearly shown in figure 6.1.3, the primary service type being ON_DEMAND services totalling over a million offers of this type. This reflects the commercial demands of cloud

computing, highlighting the need for on demand services. Due to diversified datasets, a small percentage of 1.38% of ingested services came up with mapping errors meaning a canonicalised type could not be selected for these.

These ingestion results clearly highlight both the strengths and weaknesses of CloudCompare’s ingestion pipeline, demonstrating the capabilities of the pipeline, especially when handling large quantities of data, achieving fast ingestion speeds and relatively low error and failed ingestion rates. These results also highlight potential improvements required in the canonicalisation framework, to ensure higher mapping rates and less service data disregarded.

6.2 API Performance Results

The API is a critical component, as an intermediate for all data transfer and processing between server and client, the performance directly affects user experience. Due to this, the API is evaluated based on performance during a number of conditions, measured in latency response time (ms). Endpoints were categorised and tested multiple times to compute an average response time, which is used to identify the impacts of specific parameters or conditions on response time, locating bottlenecks.

6.2.1 Figure Endpoint Response Times

Category	Endpoint	Avg Response Time (ms)	Comments
Core Retrieval	/*	86.75ms	Core endpoints require limited computation.
	/services	85ms	Simple list retrieval
	/admin/status	48ms	Small DTO payload
	/login	165ms	Authentication overhead
	/saved	49ms	Simple data retrieval
Offer Filtering		298.5ms	Filtering across large datasets
	/offers?provider=AWS	418ms	Large data set
	/offers?provider=GCP	385ms	Large data set

	/offers?region=UK	249ms	Indexing applied to region
	/offers?type=DATA_TRANSFER	142ms	Smaller dataset improves performance
Searching & Sorting		1863.3ms	Overall, longer response times due to searching
	/offers?q=BLOB	2502ms	Individual field comparisons, more time consuming
	/offers?orderBy=provider	610ms	Sorting required increases time over basic retrieval.
	/offers?orderBy=Price	2478ms	References separate the dataset, requiring joining increases overhead.
Comparison	/compare/{id}?limit=50	165ms	Set limit improves response time.
AI	/compare/ai/{offerId}?offer2Id={offerId2}	12,102ms	These endpoints are computationally intensive due to design and external third party API usages.
Data Aggregation		~	Aggregation and summarisation of large datasets is often computationally intensive
	/offers/map	1358ms	Aggregation of large dataset
	/offers/dt	418ms	Smaller dataset enables faster aggregation
	/emissions/map?serviceId=svc-storage	614ms	Filtering reduces data output and requires aggregation.
	/emissions/map?serviceId=svc-compute	10,204ms	Aggregating and computation of 1M+ service offers

While response times vary across endpoints, core functionality remained optimised, with basic structured retrieval averaging response times of under 100ms. This is due to the nature of the data, utilising basic table look ups on small datasets with no aggregate or advanced filtering being required. Endpoints utilising filtering averaged higher latency experiencing 300ms on average, due to larger datasets and generic filters being applied, with more specific filters reducing response time, comparisons returned similar response times, in aid of parameterised limits to reduce payload, increasing performance. Advanced parameter options, such as query-based searching and ordering drastically increased response time to averages nearing 2000ms due to the multi-table queries and sorting algorithms on large datasets. Furthermore, data aggregation shows varying response times, dependent on dataset size and optional filtering settings. AI endpoints, relying on third party requests and the model runtime noticeably increased response times, while latency is drastically higher than other endpoints, the tool is designed supplementary and not intended as a fundamental feature, compromising performance and speed but providing meaningful insight.

Figure 6.2.2 Parameterised queries visualisation

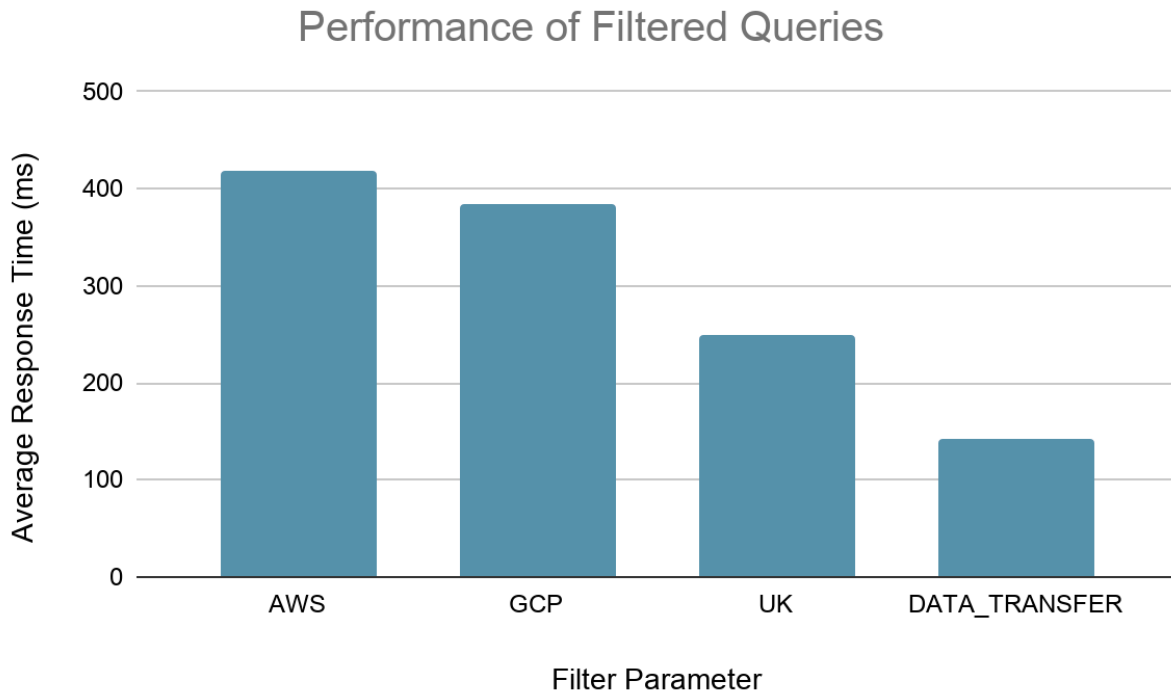


Figure 6.2.2 exhibits response times across a variety of parameterised filter queries, these queries range from 140ms to 420ms. Performance is dependent on filtered data set size, demonstrating the importance of appropriate filtering to reduce processing power as well as response times. This figure demonstrates larger datasets such as AWS have almost 4 times greater response times than smaller data sets such as data transfers, this justifies the use of both filter-based querying as well as supporting the canonicalisation model to reduce redundant information. These techniques help to improve performance, scalability and maintainability of the fundamental data model behind the CloudCompare platform.

6.4 Environmental Model

The environmental model is evaluated on how the model produces carbon estimates based on differing variables, ensuring that the model proportionally adjusts carbon emissions as expected across services with a variety of providers, service types, regions. This technical evaluation ensures that produced emission data aligns with published carbon emission trends.

Table 6.4.1 Emissions Sample Table.

Provider	Service Type	Region	Co2(kg/h)	Rating
AWS	Storage	EU-NORTH	0.003	A
AWS	Database	ASIA-SOUTH	0.133	D
AZURE	Compute	AFRICA	0.106	D
AZURE	Storage	ASIA-EAST	0.034	B
GCP	Compute	US-EAST	0.067	C
GCP	Database	EU-NORTH	0.008	A

Figure 6.4.1 presents a sample of calculated emission data, generated by the carbon emission model. This data reflects estimated carbon emissions and ratings with varying service types and providers in a selection of regions. The data demonstrates how slight changes in variables can alter emission output, in the aim of carbon emission analysis, providing understanding of the driving factors in cloud-based emissions.

These results clearly indicate proportional and expected differences in produced emissions, dependent on variables confirming the output of valid and predictable results from the model.

Figure 6.4.2 Provider-Service Average Co2 Emissions

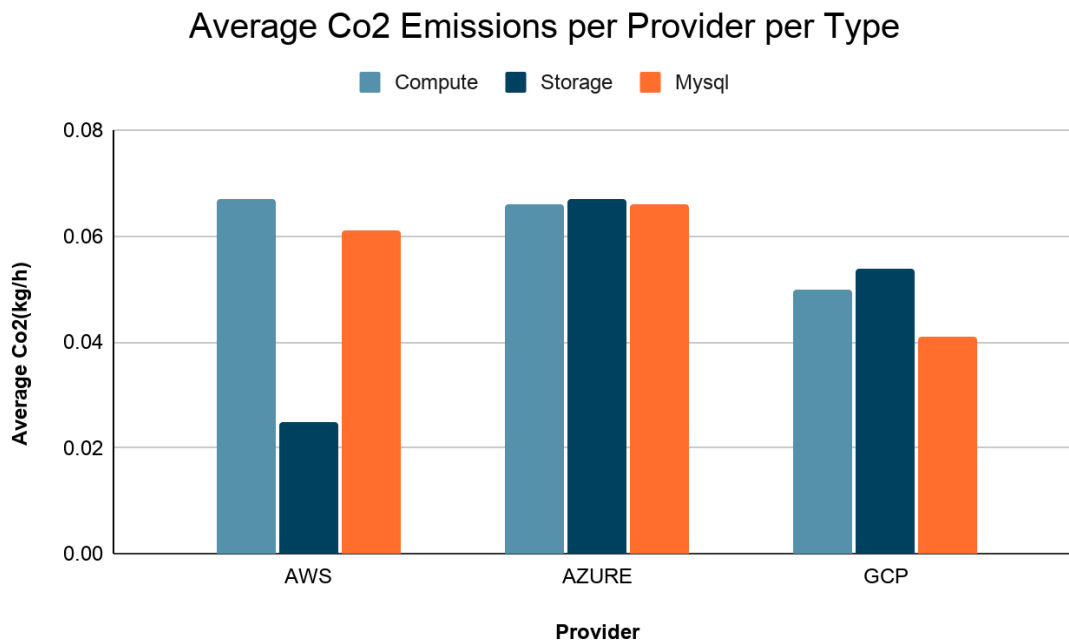


Figure 6.4.2 illustrates aggregated emission data, averaging emissions across regions to produce provider-centered emissions. This indicates that Azure produces higher overall average emissions than other providers, with GCP displaying lower emission averages across their available services and regions. These averages are weighted by the distribution of provider region availability, lower emissions suggest available carbon intensive regions being offered for services, explaining the much lower emissions exhibited by AWS Storage services. This Visualisation is useful for providing an abstract provider-based overview on carbon emissions, however, exposes true region distribution in the dataset and is not indicative of true provider emissions.

Figure 6.4.3 Average Emissions by Region

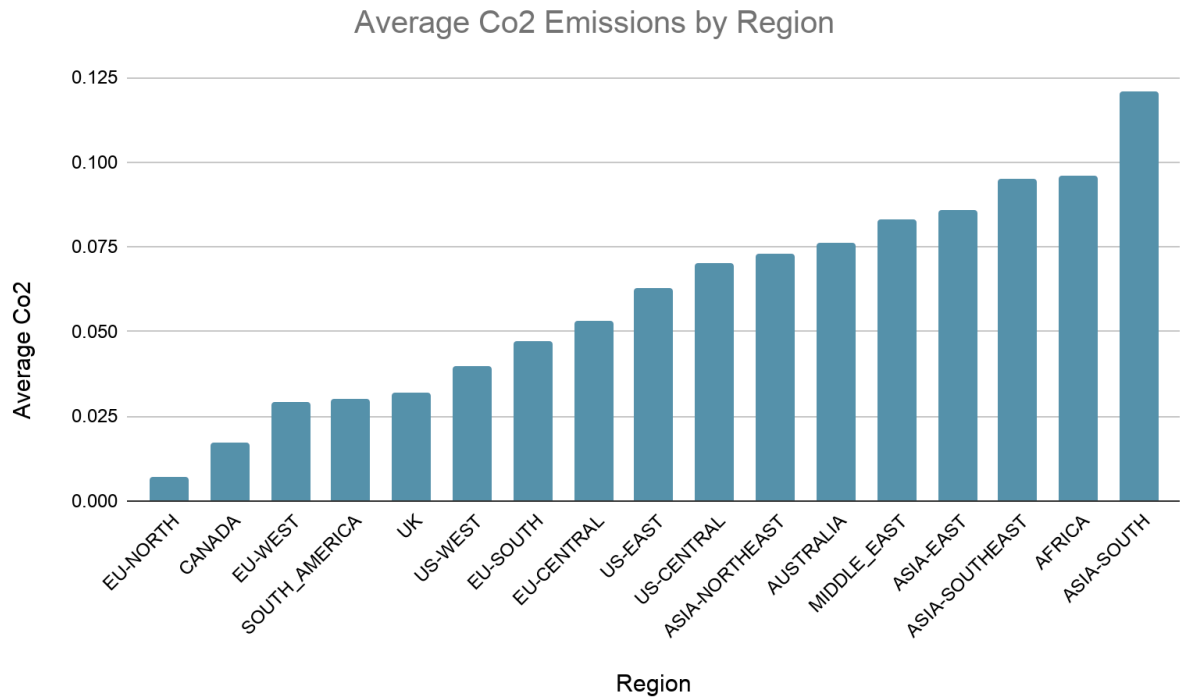


Figure 6.4.3 demonstrates that the effectiveness of the environmental model in its ability to generate emission estimates proportional to factors such as regional carbon intensity. This is shown with regions such as EU-NORTH and CANADA producing the lowest average carbon emission, in line with publicly available emission data and acknowledged regional emission trends. This figure is significant as it indicates the model is proportionately responding to its core variables, providing meaningful insight into the regional and environmental considerations that are so critical to cloud service migration.

Figure 6.4.4 Average Emission Map Visualisation

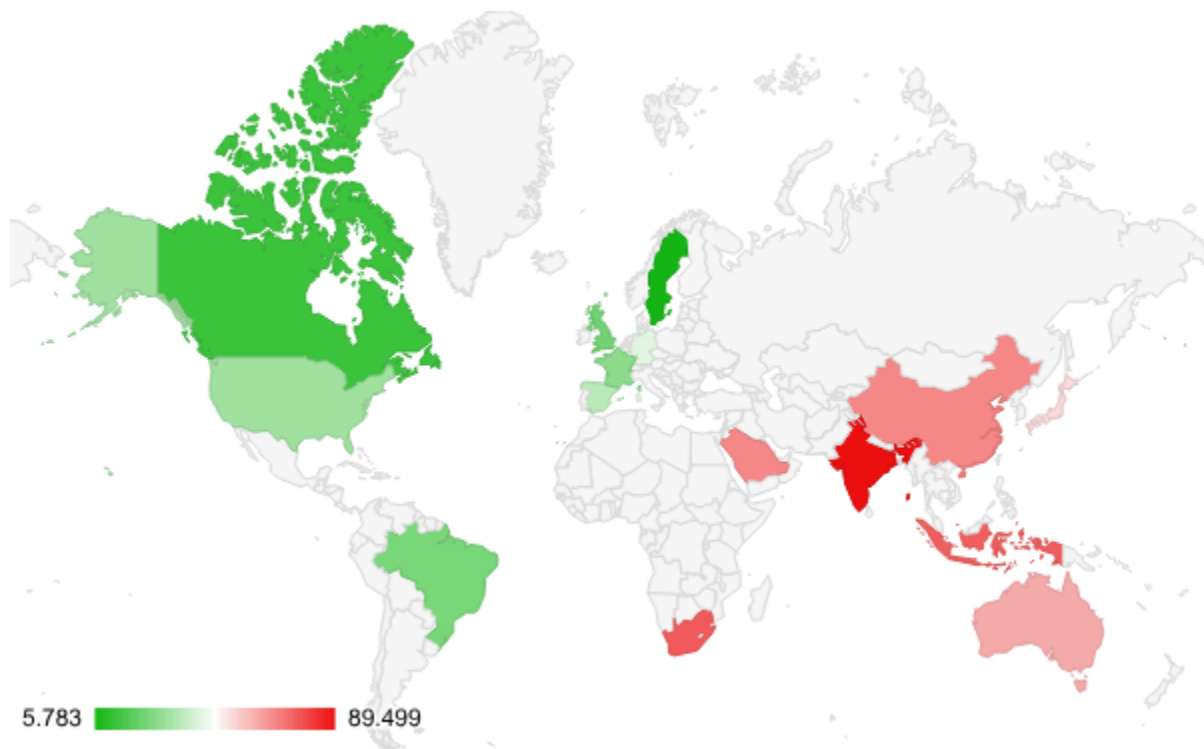


Figure 6.4.4 provides a visualisation of produced emission data, mapping average regional emissions calculated by the model, this helps to visualise the relevance of region choice in migration considerations and highlight key geographical areas that produce higher emissions. Low emission areas, those with higher renewable energy sources, are mapped in green with high emission regions being mapped in red. Data has been mapped onto representative countries associated with regions, rather than accurate data and the visualisation serves as a high level overview of regional emission results produced by the model.

6.5 User Study Results

The user study was carried out with a total of eight participants across a range of experience and technical fields such as computing, data science and cloud service specialists. Responses were measured as scores from 1-5 indicating levels of agreement; 1 = Strongly Disagree, 2 = Disagree, 3 = Neutral, 4 = Agree, 5 = Strongly Agree.

The following figure displays average feedback scores across different components of the user study.

Figure 6.5.1 Mean Evaluation Scores

Category	Included Tasks	Average Score
Core interaction	Service Location, Filtering, Saving	4.53
Comparison	Alternative service, service logical, alternative trust.	4.43
AI Compatibility	AI clarity, understanding and trust.	4.4
Visuals	Dashboard clear, useful, transfers located, map usability	4.38

Figure 6.5.1 demonstrated the mean scores across all likert categories of the study, indicating that all sections received high results between averaging between Agree and Strongly Agree. This shows that the system is perceived as usable, with high clarity and trust of the provided system services.

Figure 6.5.2 Likert feedback scores

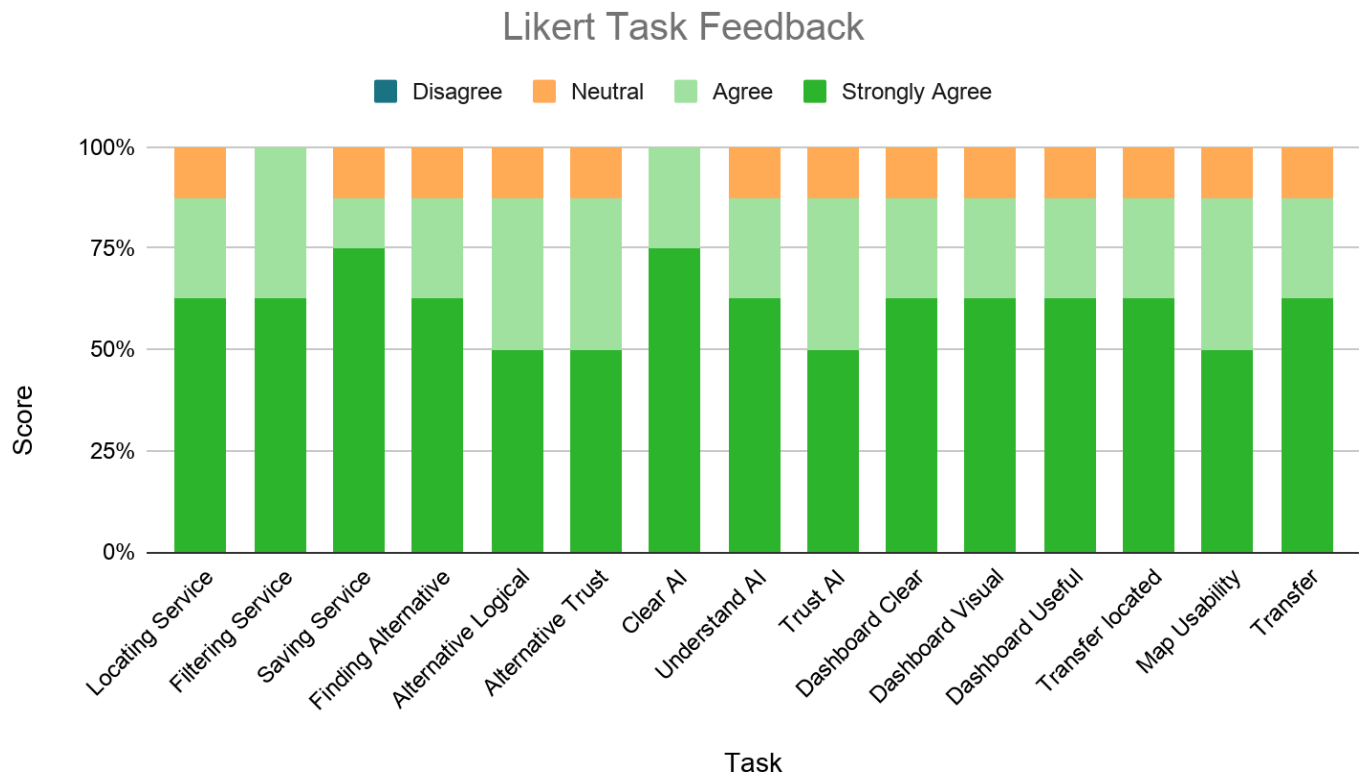


Figure 6.5.2 is indicative of task feedback for the user study, highlighting the distribution of feedback scores across each component of the study. The figure clearly shows a large distribution of Agree and Strongly Agree feedback, with few neutral opinions and no disagreements on any of the questions. This reiterates the high averages shown in figure 6.5.1, confirming the success of the study, validated by usability scores. Fundamental tasks such as service, location, filtering and saving, indicating that navigation was accessible and intuitive.

6.5.1 Qualitative Feedback

Qualitative feedback was gathered through open-ended questions, highlighting strengths of the platform, how it compares to alternatives and what features lacked clarity.

Feedback indicated there several well received features, the AI compatibility module, was highly praised for its meaningful comparative reports and semantic scoring, with one participant stating that it provided “adequate detail and explanation to allow the user to make an informed decision regardless of their pre-existing level of knowledge”. Participant feedback indicated praise for AI implementation and its role in the comparison framework, supported by the strong likert feedback scores.

Visual components of the platform were additionally considered strong by participants, with participants claiming the usefulness of data transfer visualisation “very useful at helping me understand visually where the expensive data routes are”, also noting that “the map and dashboard features stood out as incredibly beneficial for understanding and visualising the services”. This helped confirm the relevance and importance of visual aids when handling large quantities of complex data.

Additionally, certain areas were marked for improvement, especially concerning the clarity of the interactive map, claiming labels and visualisation arrows could be improved suggesting “the price labels are too small/not legible enough”, while certain visual components were “confusing.. at what appears to be random spacings”. Additionally service identification was highlighted as an area for improvement, due to the similarity with other service offers.

Conclusively and critically for evaluation, feedback regarding the meaningful support for cross-provider comparison and migration was collected, to ensure the platform met its fundamental objectives. Participants noted that the system provided “very direct and specific recommendations which otherwise would have been generic and thus inaccurate if sought elsewhere”, with further feedback stating that the platform “allows for effective comparison of features and cost”.

7. Conclusion and Discussion

The project set out to create a multi-provider cloud comparison platform, providing migration insights, service statistics and data-driven visualisations, to support the challenging process of cross-provider cloud migration across the three giants of the cloud computing industry; AWS, Azure and Google Cloud. The project aimed at solving the issues caused by fragmented service and pricing data, aiming to create one unified model to represent cloud data across multiple providers.

The system met its final objectives, providing a fully functional ingestion pipeline including data normalisation, canonicalising and storing large quantities of cloud service and pricing data. This structured data model provided the foundation upon which the comparison model was built, providing a hybrid system, enabling relevant comparisons to be made, supplemented by semantic scoring and AI-generated reports, helping give insight into the comparisons made and the difficulties surrounding their potential migration.

The results clearly indicated the success of the tested components, highlighting their importance in the CloudCompare ecosystem. In addition to this, the user study further evaluated the success of the CloudCompare platform, being well received as both a technologically correct platform

but a user-friendly platform too. To conclude, the CloudCompare platform accurately completes its purpose in providing understandable and comprehensive cross-provider cloud migration analysis and evaluation.

8. References

docs.aws.amazon.com. (n.d.). Using the AWS Price List Query API - AWS Billing. [online] Available at:
<https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/using-price-list-query-api.html>.

AWS (2019). Pricing. [online] Amazon Web Services, Inc. Available at:
<https://aws.amazon.com/pricing/>.

Amazon (2023). AWS Cloud - Amazon Sustainability. [online] Aboutamazon.com. Available at:
<https://sustainability.aboutamazon.com/products-services/aws-cloud>.

cloud-carbon-footprint.github.io. (n.d.). Methodology | Cloud Carbon Footprint. [online] Available at: <https://www.cloudcarbonfootprint.org/docs/methodology/>.

Docker (2019). Docker Documentation. [online] Docker Documentation. Available at:
<https://docs.docker.com/>.

US EPA, O. (2020). Emissions & Generation Resource Integrated Database (eGRID). [online] US EPA. Available at: <https://www.epa.gov/egrid>.

Anon, (n.d.). FinOps Framework Overview. [online] Available at:
<https://www.finops.org/framework/>.

ghgprotocol.org. (n.d.). Scope 2 Guidance | GHG Protocol. [online] Available at:
<https://ghgprotocol.org/scope-2-guidance>.

index (2025). Cloud Billing API | Google Cloud Documentation. [online] Google Cloud Documentation. Available at: <https://cloud.google.com/billing/docs/reference/rest> [Accessed 24 Apr. 2026].

Google (2025). GCP Pricing. [online] Google Cloud. Available at:
<https://cloud.google.com/pricing>.

Datacenters.google. (2025). Blocked. [online] Available at: <https://datacenters.google/efficiency/>.

Holori. (2023). Holori - The Best Cloud Calculator. [online] Available at:
<https://holori.com/cloud-calculator/>.

Holori (2025). Holori Cloud Calculator. [online] Holori.com. Available at: <https://calculator.holori.com/compare> [Accessed 24 Apr. 2026].

IEA (2025). Electricity 2025 – Analysis - IEA. [online] IEA. Available at: <https://www.iea.org/reports/electricity-2025>.

Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. [online] arXiv. Available at: <https://arxiv.org/abs/1908.10084>

Hugging Face (2026). sentence-transformers/all-MiniLM-L6-v2. [online] Available at: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

Sentence Transformers (2026). Sentence Transformers Documentation. [online] Available at: <https://www.sbert.net/>

9. Appendix

9.1 Proposal

Cloud Service Mapping & Cost Analysis Proposal

Abstract

This project proposes the development of a Cloud service mapping & comparison application which uses data gathered from cloud provider APIs to analyse and compare cloud services. A cost comparison engine will be created and used to compare performance, resource efficiency, and other properties to identify and evaluate the best-suited service as fit to user requirements. This comparison engine would consider environmental factors, such as carbon emissions, to help the end-user make informed decisions when considering migration.

In addition to cost analysis, this project will compare and verify the compatibility of services across providers, ensuring version compatibility and equivalence in the aid of smooth host migration. To achieve this, the system will make use of Natural Language Processing (NLP) to compare the similarity and equivalence of these services, providing a score and ranking of the compatibility of other potential services. The system will also make use of an organised database of metadata about services in order to make a fair comparison and to rank services based on their efficiency and compatibility.

The outcome of this project will be a data-driven system which will give the user tailored recommendations based on a range of different factors when migrating hosts, presented to the user through an interactive dashboard to help visualise all available options and recommendations generated by the engine.

1. Introduction

With the large adoption of cloud computing across the globe, multi-cloud solutions are becoming increasingly popular as clients and organisations seek greater flexibility as well as opportunities provided by the diverse range of cloud providers. The use of multi-cloud solutions has increased in recent years, with as many as 89% of companies using multi-cloud solutions in 2024. (Flexera, 2024).

However, with the increase in cloud computing usage, there come new difficulties managing workloads as environments become ever more complicated. With such a wide range of services available from each provider, it can become difficult to compare and make it especially hard to check service compatibility and equivalence between providers while still considering the most cost-effective option. Due to the wide range of variables and pricing options, it can be an overwhelming task for many organisations and individuals. As well as this, many organisations

carefully consider their environmental impact. It is important to many organisations to try and reduce their environmental footprint, with the climate impact of data centres expected to increase due to rising demand as well as an increase in the carbon footprint by 13% between 2020 and 2030 (Aslan et al., 2024).

This project aims to address these issues, focusing on the creation of a service compatibility engine utilising a large database of cloud service metadata, aided by the integration of Natural Language Processing (NLP) to evaluate and compare different services and their descriptions. This aids in supporting a seamless migration process by finding equivalent and suitable services on other providers while still maintaining financial optimisation through the use of an integrated cost-comparison engine. Furthermore, the system will incorporate sustainability metrics such as CO₂ emissions in order to help evaluate the environmental impact of each service across providers. This allows users to help make informed migration decisions which are both financially and environmentally responsible.

2. Background

With the rapid growth of cloud dependence on modern-day infrastructure, many complications and challenges extend past performance, such as environmental impact, which is a growing issue as our world modernises. Recent research and studies show that data centres account for around 1-1.5% of global electricity use, with usage growing 20-40% annually, showing the increased demand for data-driven cloud computing in our modern age. (International Energy Agency). This rapid growth shows cause for concern, especially with recent findings suggesting that emissions from data centres of Google, Microsoft and Meta may be up 7.62 times higher than officially reported. (The Guardian, 2024). The global data centre industry is expected to emit 2.5 billion metric tons of carbon dioxide by 2030 (Reuters, 2024). Sustainability is a crucial consideration in all forms of cloud computing, especially when evaluating cloud service choices and potential migration.

Commercial software such as VMware's CloudHealth is a good example of relevant financial analysis of cloud computing. Providing customisable dashboards and economic reports this software helps clients' budget and optimise their deployments. This software also provides environmental insights with an in-depth report on carbon emissions as well as power consumption, offering insights into energy usage and presenting them in understandable ways to non-specialist users. However, while a robust software for financial management and sustainability reporting, this software lacks the fine-grained version compatibility and service equivalence engine proposed in this engine. Despite this, CloudHealth remains a well-rounded, refined piece of software which provides industry-grade financial ops, however shows there is a gap in the market for migration compatibility checking.

Much like CloudHealth, Holori is another example of a high-standard financial analysis platform, collecting data from AWS, Google and Azure to generate in-depth dashboards. It provides cost tracking and asset inventory tools, providing economic and cloud optimisation advice. However, much like CloudHealth, Holori prioritises financial analysis rather than service equivalence or

in-depth migration planning. These limitations in commercial software suggest a priority in financial analysis, leaving a gap for cross-provider comparison and service equivalence support required for effective cloud migration.

Open-source projects present good examples of cost comparison and cross-benefit analysis in cloud computing. One example being “dbcost” by Bytebase, which gives a strong cost comparison across a multitude of database services from providers with different specs, pricing and location. However, due to its open-source nature, this software is limited, supporting only database services with weaker analysis and less depth. This tool also fails to support Natural Language Processing and has a few user-defined constraints, making it less suitable for the end user. While “dbcost” provides a strong benchmark for database service prices from a range of providers, it lacks the equivalence mapping and proper migration tools proposed for this project.

When considering service comparison and equivalence checking, this project will integrate Natural Language Processing (NLP) to analyse and compare cloud service descriptions across providers to find equivalent and compatible services. There are other approaches, such as keyword matching or database comparison; however, these come with limitations and lack the accuracy required by this project. Due to this, this project will make use of Sentence-BERT, which improves upon the existing BERT model, increasing efficiency and allowing the proposed comparison engine to understand the exact meaning of each service and compare them appropriately. This Sentence-BERT model was first proposed by Nils Reimers and Iryna Gurevych (2019). This model takes text and produces numerical values which can be compared using cosine-similarity. Due to this, it helps to reduce the time needed to find the most similar pair from 65 hours with BERT to around 5 seconds with SBERT. This method will be implemented in my project and used to compare and evaluate the equivalence of similar service descriptions, such as “Amazon Simple Storage Service (S3)” and “Azure Blob Storage”, which will produce a similarity score.

3. Proposed Project

3.1 Aims and objectives

This project aims to design and create a multi-cloud service mapping and comparison application, which will help users to identify similar or equivalent services offered by a range of providers, allowing them to make informed decisions based on financial, environmental and compatibility data. This project aims to help simplify and aid with the often tricky process of cloud migration, giving users access to an interactive dashboard designed to aid them in making an informed decision. This project aims to collect and compare services provided by the three major cloud providers; Amazon Web Services, Microsoft Azure and Google Cloud Platform.

These are the following objectives required to meet the aim set out for this project.

- **Develop a cloud data collection system** - This would involve creating an efficient backend system built in Java to collect all of the respective service and pricing information from each service provider. This data would be collected directly from cloud provider APIs (AWS, Azure and Google Cloud). This data will be used in the cost

analysis engine as well as the service equivalence matching engine proposed in this project.

- **Design a suitable database for cloud data** - A suitable and organised database will be designed and created in order to store this collected metadata. This data will be properly stored and formatted to maintain consistency with data gathered from different sources and cloud providers, crucial for giving a fair and consistent analysis across providers.
- **Develop a cost-analysis engine** - A cost-analysis engine will be developed to compare services across providers based on their pricing and performance to give an appropriate ranking, providing a fair cost-benefit analysis.
- **Develop a service equivalence matching engine** - A service equivalence matching engine will be used to compare equivalent services based on metadata, with the aid of Natural Language Processing to identify similar and compatible services based on their versions and descriptions.
- **Designing the interactive user dashboard** - A web-based dashboard will be designed to aid in the visualisation of the collected and processed data. This dashboard will allow users to directly compare services and view their compatibility, in addition to being presented with the cost-benefit analysis of a range of services.
- **Integrating additional sustainability statistics** - Environmental statistics will be implemented in the dashboard to help give users insight into their carbon footprint, and allow them to consider alternative, potentially more eco-friendly services or providers.
- **Ensuring performance and optimisation** - The system will be highly optimised, ensuring that data is retrieved quickly and efficiently, making use of caching and other optimisation techniques.

4. Methodology

The methodology of this project will be an organised approach to implementing the framework of this project, focusing initially on the design and efficient data collection, ensuring strong foundations that can be built upon with the proposed data analysis engines and interactive dashboards. The project will be developed layer upon layer, making use of iterative testing throughout to ensure the effectiveness of the project. The latter part of the project will focus more on frontend development and optimisation to ensure validity and consistency throughout the project.

4.1 System Architecture

- **Backend System** - The backend system will be built in Java, responsible for all data gathering and processing, calling upon provider APIs to gather data before cleaning, organising and storing this data suitable for further analysis through the use of cost and compatibility engines. Java is suitable for the backend due to its performance, high-level support, as well as its support for NLP frameworks and API usage.
- **Database System** - The database will be crucial for this project in order to normalise and format all of the collected data from the range of providers. The data must be properly structured for effective querying. This data will contain information on providers,

services and pricing details. The chosen database for this is either PostgreSQL or MySQL.

- **Frontend System** - The frontend system will be in the form of an interactive dashboard, allowing the user to view all of the processed data in a neat and organised way. This dashboard will contain charts and other visual elements to help with the analysis of data. This front-end system will be created in JavaScript with help from additional frameworks.

4.2 Data Collection

Due to this project being data-driven, the data collected must be reliable and effectively collected to create a solid foundation for further analysis. The majority of the data will be collected directly from the public provider APIs. These APIs provide a wide range of consistent and comparable data, which will be incredibly useful for analysis and comparison. These are the following APIs that the project shall use

- **AWS** - AWS Cost Management API, AWS Resource group tagging API, AWS Service Catalogue
- **Azure** - Azure Resource Graph
- **Google** - Catalogue API, Asset Inventory API

These APIs are incredibly in-depth with substantial documentation and support, making them suitable sources of data for my project. This data contains: lists of services, prices of services and other useful metadata.

4.3 Service Comparison Engine

The service comparison engine will make use of semantic word matching approaches with the use of a Natural Language Processing model. Sentence-BERT will be implemented to compare names, versions and descriptions of services to produce a compatibility score, which can rank services based on their compatibility. This adds a layer of complexity that is often needed to ensure proper equivalence when migrating important services. These comparison rankings will be considered by other parts of the program and will be used to make appropriate migration suggestions.

4.4 Cost Evaluation Engine

The cost evaluation engine will make use of all available and formatted data to help make informed decisions when comparing the prices of services based on factors such as region, performance and available services. In addition to this, the engine will also consider sustainability factors, providing insights on carbon footprint, estimating CO₂ emissions based on service provider information.

4.5 Frameworks

My project will make use of a variety of frameworks to properly meet all objectives and aims set out.

- **Languages:** Java will be used for the backend, with JavaScript being used for the frontend of the project.

- **Frameworks & Modules:** React will be used for the frontend UI design. ONNX will be used for the Sentence-BERT model's implementation in Java. Spring will also be used for API interaction on the backend.
- **Database:** MySQL or PostgreSQL will be used for the backend databases due to their relational properties.
- **Version Control:** GitHub will be used for effective version control.

5. Programme of work

This is a breakdown of the stages of the project.

- **Weeks 1-3 Research and Proposal**
 - Complete initial research into existing cloud cost comparison research papers and existing tools
 - Look into potential frameworks, libraries and APIs that could be useful to the project
 - Identify project aims and objectives, completing the project proposal.
- **Weeks 4-5 Planning and Database design**
 - Finalise plans for system layout, with clear architecture for frontend and backend systems.
 - Get familiarised with APIs and respective documentation to understand the data format.
 - Finalise the database design for storing all of the collected data.
- **Weeks 6-11: Develop backend and main functionality**
 - Develop backend data collection, efficient API data retrieval and processing
 - Implement database support into the backend, allowing for collected data to be processed and inserted into the database.
 - Start development of the cost-comparison engine, analysing data and presenting it in a suitable format.
 - Implement NLP in aid of the service-comparison engine to allow comparison of service descriptions and metadata, ensuring output is appropriate and accurate for an equivalence ranking system.
 - Complete progress report in **Week 10**
- **Weeks 12-15 Frontend System**
 - Start the development of the interactive dashboard for the frontend system
 - Interact with the backend system via APIs to gather all relevant cost and comparison data
 - Ensure data is customisable and presentable on the front end
 - Make use of iterative testing to ensure effectiveness.
 - Complete progress report in **Week 15**
- **Weeks 16-18 Draft report**

ID	Task Name	2025			2025			2026
		09	10	11	12	01	02	03
1	Research and Proposal							
2	Planning and Database design							
3	Backend and main functionality							
4	Frontend System							
5	Draft Report							

6. Resources required

- **Hardware** - Hardware for the development, testing and running of the NLP will be required; however, there aren’t many hardware requirements for this, and any personal computer with 16GB RAM should do.
- **Cloud Access** - Access to the cloud providers is crucial for this project, and the student plans for AWS, Microsoft and Google will be used in order to gather data.
- **Frameworks** - Development frameworks such as **Spring Boot**, **React JS**, **GitHub** will be required for backend, frontend and version control.
- **Libraries** - ONNX Runtime will be used to support NLP.

7. References

Luxner, Tanner. “Cloud Computing Trends: Flexera 2024 State of the Cloud Report.” Flexera Blog, 28 Mar. 2024,
www.flexera.com/blog/finops/cloud-computing-trends-flexera-2024-state-of-the-cloud-report

Aslan, T., Holzapfel, P., Stobbe, L., Grimm, A., Nissen, N.F. and Finkbeiner, M. (2024). Toward climate neutral data centers: Greenhouse gas inventory, scenarios, and strategies. iScience, [online] 28(1), p.111637. doi:<https://doi.org/10.1016/j.isci.2024.111637>.

“VMware Tanzu CloudHealth | VMware Tanzu.” Vmware.com, 2025,
www.vmware.com/products/app-platform/tanzu-cloudhealth.

International Energy Agency. "Data Centres and Data Transmission Networks." IEA, 11 July 2023, www.iea.org/energy-system/buildings/data-centres-and-data-transmission-networks.

O'Brien, Isabel. "Data Center Emissions Probably 662% Higher than Big Tech Claims. Can It Keep up the Ruse?" The Guardian, 15 Sept. 2024, www.theguardian.com/technology/2024/sep/15/data-center-gas-emissions-tech.

Reuters. "Global Data Center Industry to Emit 2.5 Billion Tons of CO2 through 2030, Morgan Stanley Says." Reuters, 3 Sept. 2024, www.reuters.com/markets/carbon/global-data-center-industry-emit-25-billion-tons-co2-through-2030-morgan-stanley-2024-09-03/.

Reimers, Nils, and Iryna Gurevych. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. 27 Aug. 2019.

9.2 User Study

9.2.3 Consent statements

I confirm that I am 18 years of age or older.

I confirm that I have read and accepted the terms of the Participation Information Sheet.

I have completed the consent form and I voluntarily consent to participation in this study.

9.2.4 Study link

Study takes place on a microsoft forms link, questions in form of likert responses as well as those of a qualitative nature.

 [CloudCompare User Evaluation Form – Fill in form](#)